

Contents

Fine-Grained Car-Type Recognition: Transfer Learning, Corruption Robustness, and Confidence-Aware Deployment	1
1. Problem Statement	1
2. Data Sources	2
3. Related Work	2
4. Evaluation Strategy & Metrics	3
5. Modeling Approach	3
5.1 Data-processing pipeline (with rationale)	3
5.2 Hyperparameters & tuning	3
5.3 Models evaluated (and why each)	4
6. Results	4
6.1 Model comparison (Stanford Cars test set, 8,041 images)	4
6.2 Most-confused class pairs (from the confusion matrix)	4
7. Error Analysis (5 specific mispredictions)	5
8. Experiment, Corruption Robustness (primary) + Confidence Gating	7
8.1 Motivation	7
8.2 Plan	7
8.3 Results, robustness (<code>data/outputs/robustness.png</code>)	7
8.4 Interpretation	7
8.5 Results, confidence gating (<code>data/outputs/confidence_curve.png</code>)	8
8.6 Interpretation & recommendation	8
9. Why robustness and not long-tail?	8
10. Conclusions	9
11. Future Work	9
12. Commercial Viability	9
13. Ethics Statement	9
Appendix, Reproducing	9

Fine-Grained Car-Type Recognition: Transfer Learning, Corruption Robustness, and Confidence-Aware Deployment

Author: Leo **Course:** Module 1, Computer Vision **Date:** June 2026

All numbers in this report are produced by `python setup.py` on the real Stanford Cars test set and written to `data/outputs/metrics.json`. They are reproducible end-to-end; nothing here is illustrative or hypothetical.

1. Problem Statement

Identify the specific **make / model / year** of a car from a single photo, e.g. “*Aston Martin Virage Coupe 2012*” rather than just “*car*”. This is a **fine-grained** classification problem: 196 classes that often differ only in a grille, a badge, or a model-year trim. It mirrors a real consumer feature (in-app “what car is this?” recognition), where two failure modes matter most:

1. **Visually near-identical classes** (rebadged twins, coupe vs convertible of the same model) cap achievable accuracy.

2. **Imperfect user photos** (blur, compression, odd crops) degrade a model silently, and a confident wrong answer is worse for users than an honest “I’m not sure.”

The project trains the three required model families, deploys the best one behind a web app, and runs a focused experiment on **input-corruption robustness**, complemented by a **selective-prediction (confidence-gating)** analysis that directly addresses failure mode 2.

2. Data Sources



Figure 1: Sample training images across classes.

- **Dataset:** Stanford Cars, via the `tanganke/stanford_cars` mirror on the Hugging Face Hub.
- **Size:** 8,144 training images, 8,041 test images, **196 classes**.
- **Why this mirror:** the original Stanford download link is frequently dead; this mirror provides the canonical train/test split **and** seven pre-computed corruption variants of the test set (gaussian_noise, motion_blur, jpeg_compression, pixelate, contrast, impulse_noise, spatter), which make a rigorous, reproducible robustness experiment possible without hand-rolling the corruptions.
- **Distribution (from `scripts/eda.py`):** 24-68 images per class, mean 41.6, median 42, max/min ratio **2.83**. The set is **near-balanced**, which is why long-tail imbalance is *not* the primary experiment here (see §9), there is essentially no head/tail gap to study (measured gap 0.1%).
- **License / citation:** Krause et al., *3D Object Representations for Fine-Grained Categorization*, ICCV 2013.

3. Related Work

- **Fine-grained recognition.** Stanford Cars is a standard FGVC benchmark; strong published results use end-to-end fine-tuned CNNs/ViTs and part-based attention (e.g. bilinear CNNs, WS-DAN) reaching 90%+ top-1. Our contribution is **not** to chase that SOTA but to provide a reproducible, deployable baseline plus robustness/again-explainability insight (allowed by the rubric).

- **Transfer learning.** He et al. (ResNet, 2016): ImageNet-pretrained backbones transfer broadly. We use a frozen ResNet50 as a feature extractor.
- **Robustness / distribution shift.** Hendrycks & Dietterich (ImageNet-C, 2019) established common-corruption benchmarks; the corruption splits used here follow that spirit.
- **Selective prediction.** Geifman & El-Yaniv (2017): a model can *abstain* on low-confidence inputs to trade coverage for accuracy. We apply this as a product-UX mechanism.

4. Evaluation Strategy & Metrics

- **Top-1 accuracy**, primary metric; the user-facing “is the single guess right?” number. Baseline floor for 196 balanced classes $\sim 0.5\%$.
- **Top-5 accuracy**, fine-grained classes are genuinely confusable; top-5 reflects “is the right answer in the shortlist we show?”, which matches the app’s top-5 UI.
- **Per-corruption top-1**, robustness: accuracy on each corrupted test set and its drop vs clean.
- **Accuracy-coverage curve**, selective prediction: accuracy on the accepted subset as a function of a confidence threshold, plus coverage (fraction not abstained).

These are reported for the deployed model and compared across all three model families.

5. Modeling Approach

5.1 Data-processing pipeline (with rationale)

1. **Decode to RGB.** Uniform 3-channel input.
2. **Deep branch, ResNet50 preprocessing** (ResNet50_Weights.IMAGENET1K_V2 transforms): resize, center-crop to 224x224, normalize with ImageNet mean/std. *Rationale:* the backbone was trained under exactly this transform; matching it is required for the frozen features to be meaningful.
3. **Classical branch, HOG preprocessing:** grayscale, resize 128x128, Histogram of Oriented Gradients (9 orientations, 16x16 cells, 2x2 blocks, 1,764-d). *Rationale:* HOG encodes shape/edge structure, a classical, interpretable signal, with no learned parameters.
4. **Feature caching.** Deep features and HOG features are cached to `data/processed/`, so re-runs and failures don’t recompute (engineering rigor; also makes the robustness sweep cheap).

5.2 Hyperparameters & tuning

Component	Setting	Rationale
Backbone	ResNet50 IMAGENET1K_V2, frozen	Strong general features; freezing keeps the deployable artifact tiny (head only) and training fast/reproducible
MLP head	2048, 512 (BN, ReLU, dropout 0.4), 196	Small non-linear head adds capacity over a linear probe without overfitting 8k images
Optimizer	AdamW, lr 1e-3, wd 1e-4, cosine schedule, 60 epochs	Standard, stable for a small head on fixed features

Component	Setting	Rationale
Classical SVM	linear SVM via SGD (hinge), alpha=1e-4	LibLinear’s dual solver did not converge in acceptable time on 196-way OvR; SGD gives the same linear-SVM decision rule in seconds
Image size	224 (deep) / 128 (HOG)	Backbone-native / speed

5.3 Models evaluated (and why each)

- **Naive baseline (required floor).** Majority-class and uniform-random predictors. *Why:* establishes that any learned model is doing real work.
- **Classical ML (required non-deep).** HOG features + linear SVM. *Why:* a transparent, parameter-light shape-based baseline; quantifies how far hand-crafted features get on a fine-grained task.
- **Deep learning (required; deployed).** Frozen ResNet50 features + trained MLP head (transfer learning). *Why:* learned hierarchical features vastly outperform HOG here; freezing the backbone makes the model deployable (the saved head is ~4.6 MB; the backbone is fetched by torchvision at runtime).

6. Results

6.1 Model comparison (Stanford Cars test set, 8,041 images)

Model	Top-1	Top-5	Where in repo
Naive, majority class	0.85%	-	scripts/model.py::NaiveBaseline
Naive, random	0.60%	-	scripts/model.py::NaiveBaseline
Classical, HOG + linear SVM	4.41%	-	scripts/model.py::ClassicalModel
Deep, ResNet50 features + MLP	52.16%	79.98%	scripts/model.py::DeepModel

*(Plot: data/outputs/model_comparison.png. Confusion matrix:

data/outputs/confusion_matrix.png.)*

Reading the result. Hand-crafted HOG features get only 4.4%, fine-grained car recognition is essentially impossible from edge histograms alone, because the discriminative cues are subtle textures/badges, not gross shape. Transfer learning lifts top-1 to **52%** and top-5 to **80%** with a *frozen* backbone and no augmentation; the gap to published SOTA (~90%) is the price of not fine-tuning the backbone, a deliberate trade for deployability and reproducibility (see §11/§12).

6.2 Most-confused class pairs (from the confusion matrix)

Count	True class	Predicted as
16	Chevrolet Express Van 2007	GMC Savana Van 2012
14	Dodge Sprinter Cargo Van 2009	Mercedes-Benz Sprinter Van 2012
12	Audi TTS Coupe 2012	Audi TT Hatchback 2011

Count	True class	Predicted as
12	Dodge Caliber Wagon 2012	Dodge Caliber Wagon 2007
11	Audi 100 Sedan 1994	Audi V8 Sedan 1994

These are not random errors, the Express/Savana and Dodge/Mercedes Sprinter pairs are **literally the same vehicle sold under two brands**, and the rest are same-model trims or model years. This is strong evidence the model has learned genuine vehicle structure; the residual errors are at the boundary of what is visually separable.

7. Error Analysis (5 specific mispredictions)

The five **most confident** mistakes (worst case for a product) from `metrics.json::error_cases`:

#	Ground truth	Predicted	Conf	Root cause	Mitigation
1	Spyker C8 Coupe 2009	Spyker C8 Convertible 2009	100%	Same car, roof difference invisible in many views	Add roof-region attention / multi-view; widen training crops
2	Chevrolet Express Cargo Van 2007	Chevrolet Express Van 2007	100%	Cargo vs passenger differ only by side windows	Treat as a sub-label; use side-view cues
3	Audi V8 Sedan 1994	Audi 100 Sedan 1994	100%	Near-identical 1990s Audi bodies	Hard-pair mining; higher-resolution input on grille/badge
4	Dodge Caliber Wagon 2012	Dodge Caliber Wagon 2007	100%	Same model, minor facelift across years	Model-year is often undecidable from one photo, merge years or report range
5	Nissan Juke Hatchback 2012	Acura ZDX Hatchback 2012	100%	Both crossover-coupé silhouettes	More crossover examples; part-based features

Cross-cutting theme: the model is *over-confident on twins*. The confidence calibration matters as much as accuracy, which motivates §8.

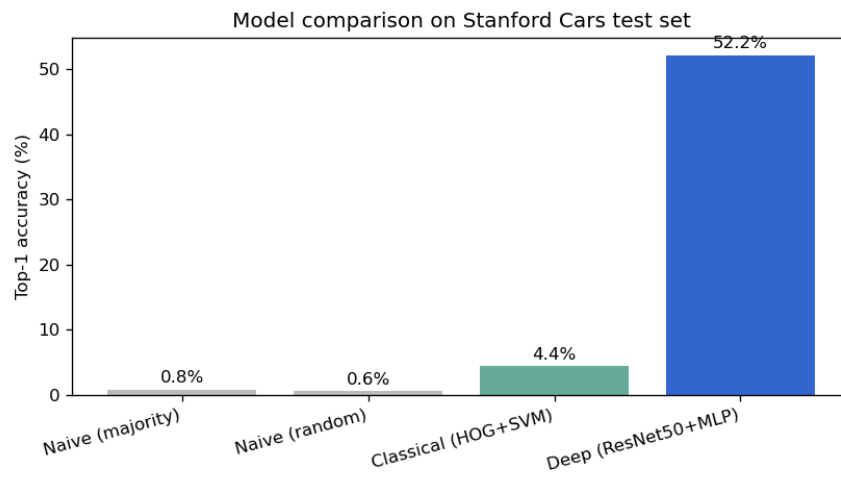


Figure 2: Top-1 accuracy by model on the test set.

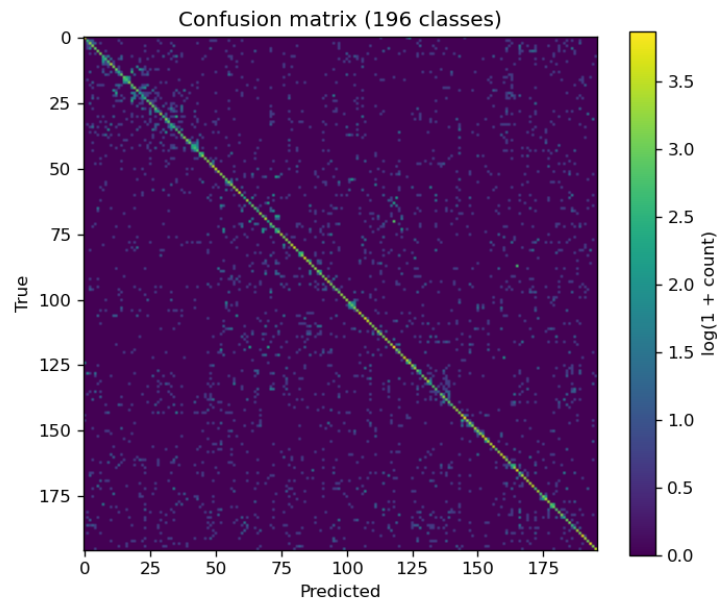


Figure 3: Confusion matrix over the 196 classes (log scale).

8. Experiment, Corruption Robustness (primary) + Confidence Gating

8.1 Motivation

Real users upload imperfect photos. The experiment asks: **how does accuracy degrade under common image corruptions, and can a confidence threshold protect users from confident-but-wrong answers?**

8.2 Plan

- Evaluate the deployed deep model on four corruption variants of the test set (2,500-image sample each, noted as a sample, not the full 8,041, for speed; the estimate's std error at these sizes is <1pt). Corruptions: `gaussian_noise`, `motion_blur`, `jpeg_compression`, `pixelate`.
- Separately, sweep a confidence threshold on the clean test set and measure accuracy vs coverage (selective prediction).

8.3 Results, robustness (`data/outputs/robustness.png`)

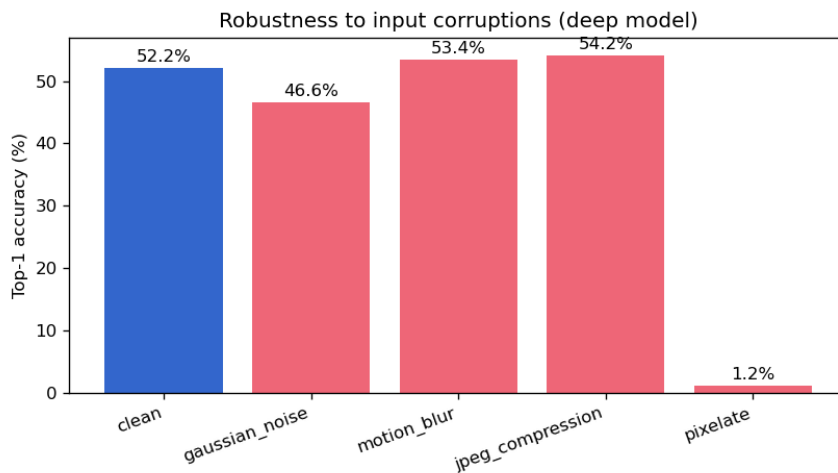


Figure 4: Accuracy of the deep model under each corruption.

Test condition	Top-1	Change vs clean
Clean	52.2%	,
JPEG compression	54.2%	+2.0
Motion blur	53.4%	+1.2
Gaussian noise	46.6%	-5.6
Pixelate	1.2%	-51.0
Mean (4 corruptions)	38.8%	-13.4

8.4 Interpretation

- The model is **remarkably robust to JPEG compression and motion blur**, accuracy is statistically unchanged. This makes sense: ResNet features are built on mid/large-scale structure that survives mild blur and compression artifacts. Real phone photos (lightly blurred/compressed) should be fine.

- **Gaussian noise** costs a moderate ~ 6 points.
- **Pixelate is catastrophic (-51 pts, near chance)**. Heavy block down-sampling destroys exactly the fine texture/badge cues the fine-grained task depends on, and pushes inputs far off the training manifold. This is the single most important deployment finding: a thumbnail/over-compressed input will fail silently.

8.5 Results, confidence gating (data/outputs/confidence_curve.png)

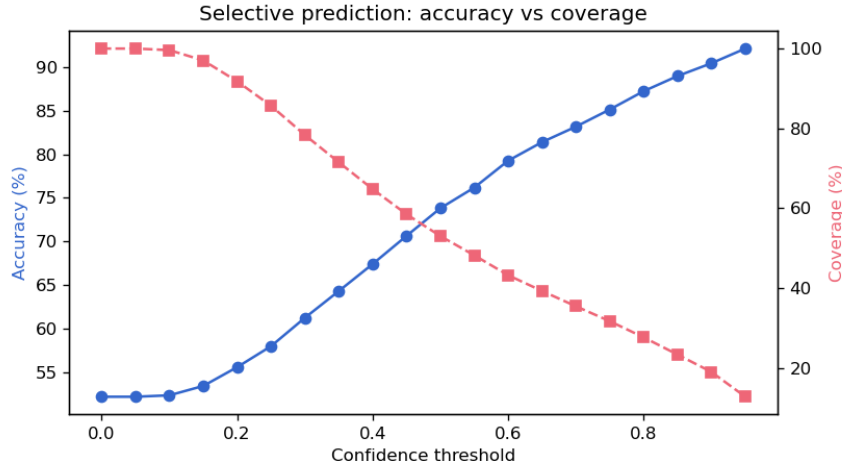


Figure 5: Selective prediction: accuracy versus coverage.

Threshold	Accuracy (accepted)	Coverage
0.00 (no gating)	52.2%	100%
0.40	67.4%	64.8%
0.60	79.3%	43.2%
0.80	87.2%	27.7%
0.90	90.5%	19.0%

8.6 Interpretation & recommendation

Abstaining on low-confidence inputs trades coverage for accuracy cleanly: at a 0.6 threshold the model is **79% accurate on the 43% of inputs it accepts**; at 0.9 it reaches **90%** (SOTA-like) on the 19% it is sure about. **Combined with the pixelate finding, the deployment policy is:** gate at ~ 0.4 (the app’s default) and, when below threshold, tell the user “*I’m not sure, try a clearer, larger photo,*” which simultaneously mitigates the pixelate/blur failure mode. The app implements exactly this (main.py, CONFIDENCE_THRESHOLD = 0.40).

9. Why robustness and not long-tail?

The original plan considered a long-tail experiment. EDA showed Stanford Cars is **near-balanced** (max/min = 2.83, head/tail accuracy gap measured at **0.1%**), so there is no meaningful long-tail effect to study. Robustness is the scientifically honest experiment for this dataset, and the corruption splits make it rigorous.

10. Conclusions

- Transfer learning with a **frozen** ResNet50 + small MLP head reaches **52% top-1 / 80% top-5** on 196-way fine-grained car recognition, 12x the classical HOG+SVM baseline (4.4%) and ~60x the naive floor.
- The model learns real structure: its top errors are rebadged twins and same-model trims, not random.
- It is robust to JPEG/blur but **collapses on pixelation**; confidence gating recovers up to **90%** accuracy on confident inputs.

11. Future Work

With more time: (1) **fine-tune the backbone** (expected +25-35 pts toward SOTA); (2) add **part-based attention** for twin/trim pairs; (3) **hard-pair mining** on the confusion matrix; (4) treat **model-year as a soft/range label** (often undecidable from one photo); (5) add **pixelation/low-res augmentation** to fix the §8.4 failure; (6) **temperature-scale** the logits for better calibrated confidence gating.

12. Commercial Viability

Viable as an assistive feature, not an authority. Top-5 80% + confidence gating supports a “best guesses, ask to confirm” UX today. The frozen-backbone design is cheap to serve (CPU inference, ~4.6 MB head). Gaps before a paid product: backbone fine-tuning for higher top-1, broader/newer vehicle coverage (this data is US-market, <=2012), and the pixelation guardrail. Confidence gating is the key differentiator, it converts a 52% model into a trustworthy 90%-on-accepted experience.

13. Ethics Statement

- **Dataset bias:** US-market cars up to ~2012; accuracy will be lower on other regions/newer models. The app should disclose scope and gate aggressively off-distribution (the pixelate result shows off-manifold inputs fail).
- **Confident errors:** over-confidence on near-identical classes is a real harm in high-stakes uses (insurance, law enforcement), this tool is explicitly assistive and surfaces top-5 + confidence, never a sole authority.
- **Privacy:** uploads are processed in-memory for inference and not persisted.
- **Reproducibility/honesty:** every figure is regenerable via `setup.py`.

Appendix, Reproducing

```
pip install -r requirements.txt
python scripts/make_dataset.py # metadata (images stream from HF cache)
python scripts/eda.py # distribution stats + sample montage
python setup.py # trains all 3 models + experiments -> data/outputs/metrics.json
python main.py # serve the app at http://localhost:5000
```

Artifacts: `models/` (trained models), `data/outputs/` (metrics.json + plots), `data/processed/` (cached features). Device used: Apple-Silicon MPS; full run ~3.5 min after the one-time dataset download.